

Cours 3 ECABD MAABOUT

Regroupement (Clustering)

I- Qu'est ce qu'un bon regroupement ?

Une bonne méthode de regroupement permet de garantir :

- Une grande similarité intra-groupe
- Une faible similarité inter-groupe

La qualité d'un regroupement dépend donc de la mesure de **similarité** utilisée par la méthode de son implémentation.

II- Structures de données

Au niveau des structures, on va utiliser 2 types de matrices :

- Matrice de données

$$\begin{bmatrix} x_{11} & \dots & x_{1f} & \dots & x_{1p} \\ \dots & \dots & \dots & \dots & \dots \\ x_{i1} & \dots & x_{if} & \dots & x_{ip} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n1} & \dots & x_{nf} & \dots & x_{np} \end{bmatrix}$$

- Matrice de distance

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \dots & \dots & 0 \end{bmatrix}$$

III- Comment mesurer la qualité d'un clustering ?

L'objectif lors d'un **clustering**, c'est de **regrouper des objets similaires** au sein d'un même groupe (**cluster**) et de **séparer les objets dissemblables** dans des groupes différents.

Mais afin d'évaluer si le **clustering** est réussi, on va devoir évaluer sa **qualité** :

- ➔ La qualité d'un clustering dépend directement de la **qualité des mesures de similarité**
 - Cette **similarité** est exprimée par le biais d'une **mesure de distance**
 - **Plus la distance entre deux objets est petite, plus ils sont semblables**
 - **Plus la distance entre deux objets est grande, moins ils sont semblables**
 - La définition de ces distances va être très différentes selon le **type** de la **variable** analysée (à détailler juste après)
 - Mais en pratique on va utiliser ce qu'on appelle une **pondération des variables**.
- C'est-à-dire accorder **plus de poids** à certaines **variables** jugées **plus pertinentes**.

IV- Types de variables

a. Intervalles (continues, réelles)

Lorsque les données sont **numériques** (âge, salaire, taille...), on va vouloir mesurer la **distance** entre 2 valeurs pour savoir si elles sont **proches** ou **éloignées**.

Mais il va tout d'abord falloir **standardiser les données** :

En effet, certaines variables n'ont pas la même **échelle** :

- Exemple : « Age » peut varier entre 0 et 100, tandis que « Salaire » lui va varier en 1 000 et 10 000.
 - Si on ne corrige pas ça, le salaire au niveau de ses valeurs va s'imposer largement par rapport à l'âge.
 - On transforme toutes les valeurs pour qu'elles soient exprimées dans la même unité statistique (**Z-SCORE**)

Comment calculer ce Z-Score ?

1. Calculer la **moyenne** de toutes les valeurs d'un attribut (ex : Moyenne des âges) noté **mf**

$$m_f = \frac{x_1 + x_2 + \dots + x_n}{n}$$

2. Calculer l'**écart absolu moyen**, c'est-à-dire qu'on regarde à quel point chaque valeur s'éloigne en moyenne de la moyenne, noté **sf**

$$s_f = \frac{|x_1 - m_f| + |x_2 - m_f| + \dots + |x_n - m_f|}{n}$$

3. A partir de **mf** et **sf**, calculer le **z-score** pour **CHAQUE VALEUR**, noté **z** :

$$z = \frac{x_i - m_f}{s_f}$$

➔ Elle dit de combien d'écarts moyen la valeur **xi** est au-dessus ou en dessous de la moyenne **mf**

Exemple :

	Age	Salaire
Personne1	50	11000
Personne2	70	11100
Personne3	60	11122
Personne4	60	11074

1. Calcul de la moyenne mf pour Age et Salaire

$$mf_{Age} = (50+70+60+60)/4 = 60$$

$$mf_{Salaire} = (11000+11100+11122+11074)/4 = 11074$$

2. Calcul de l'écart absolu moyen sf pour Age et Salaire

$$sf_{Age} = (|50-60|+|70-60|+|60-60|+|60-60|)/4 = 20/4 = 5$$

$$sf_{Salaire} = (|11000-11074|+|11100-11074|+|11122-11074|+|11074-11074|)/4 = 148/4 = 37$$

3. Calculer le z-score z pour chaque élément de chaque attribut

	Age	Salaire
Personne1	$(50-60)/5 = -2$	$(11000-11074)/37 = -2$
Personne2	$(70-60)/5 = 2$	$(11100-11074)/37 = 0.7027027$
Personne3	$(60-60)/5 = 0$	$(11122-11074)/37 = 1.2972973$
Personne4	$(60-60)/5 = 0$	$(11074-11074)/37 = 0$

b. Binaires

Une table de contingence pour données binaires est nécessaire :

	$O_2 = 1$	$O_2 = 0$	Total
$O_1 = 1$	a	b	a + b
$O_1 = 0$	c	d	c + d
Total	a + c	b + d	n

- a : nombre d'attributs pour lesquels les deux objets ont la valeur 1 (présence commune)
- b : nombre d'attributs où $O_1 = 1$ et $O_2 = 0$
- c : nombre d'attributs où $O_1 = 0$ et $O_2 = 1$
- d : nombre d'attributs où les deux valent 0 (absence commune)
- $n = a + b + c + d$: nombre total d'attributs comparés

Exemple : $O_1 = (1,1,0,1,0)$ et $O_2 = (1,0,0,0,1)$

$$a=1, b=2, c=1, d=1$$

Mesure des distances :

Coefficient d'appariement (Matching) simple :

→ $d(i,j) = (b+c)/(a+b+c+d)$ (i.e. nombre total de position dans les objets)

- Exemple : $O1 = (1,1,0,1,0)$ et $O2 = (1,0,0,0,1)$

$D(O1,O2) = 3/5$

O1	1	1	0	1	0
O2	1	0	0	0	1

Lettre	Signification	Nombre d'occurrences
a	Lorsque l'attribut O1=1 et O2=1	1
b	Lorsque l'attribut O1=1 et O2=0	2
c	Lorsque l'attribut O1=0 et O2=1	1
d	Lorsque l'attribut O1=0 et O2=0	1

$$d(i,j) = (b+c)/(a+b+c+d) = (2+1)/(1+2+1+1) = 3/5$$

Coefficient de Jaccard :

→ $d(i,j) = (b+c) / (a+b+c)$ (d qui représente le cas où les deux objets sont à 0 n'est pas pris en compte)

- $d(O1,O2) = 3/4$ (Le 3^{ème} élément est à « 0,0 » donc on ne le prend pas en compte)

Différence conceptuelle

Aspect	Coefficient d'appariement simple	Coefficient de Jaccard
Tient compte des 0 communs ?	✓ Oui	✗ Non
Utilisé quand	Les 0 sont aussi informatifs (ex : réponses à un test vrai/faux)	Les 1 représentent une présence rare ou importante (ex : attributs ou mots-clés)
Formule	$(a + d)/(a + b + c + d)$	$a/(a + b + c)$

c. Variables nominales

Une généralisation de variables binaires. Ex : Rouge, vert et bleu

Méthode 1 : Matching simple

→ m : Nombre d'appariements ; p : Nombre total de variables

○ $d(i,j) = (p-m)/p$

Méthode 2 : Utiliser un grand nombre de variables binaires

→ Créer une variable binaire pour chaque modalité

○ Ex : Variable 'Rouge' qui prend les valeurs « Vrai » (1) ou « Faux » (0)

	Taille	Couleur
O1		Blanc
O2		Rouge
O3		Rouge
O4		Vert

Qui va devenir en termes de variables binaires : (Avec des V ou F)

	Taille	Blanc	Rouge	Vert	...
O1		V	F	F	
O2		F	V	F	
O3		F	V	F	
O4		F	F	V	

d. Catégories, Ordinales, Ratio

Variables Ordinales :

Une **variable ordinale** est une variable dont les valeurs ont **un ordre logique**, mais **pas forcément des distances précises** entre elles.

Exemples :

- Niveau de satisfaction : Pas satisfait (1), Moyennement satisfait (2), Très satisfait (3)
- Taille d'un vêtement : S, M, L, XL

On sait que $L > M > S$, mais on ne peut pas dire que "la différence entre M et S" est la même que "la différence entre L et M".

Une variable Ordinale peut être **discrète** ou **continue**.

- ➔ **Attention** : L'ordre peut être important
- ➔ Peuvent être traitées comme les **variables intervalles**
 - Remplacer X_{if} par son rang **rif** app $\{1, \dots, M_f\}$
 - Ex :

Exemple :

Valeur	Rang
Pas satisfait	1
Moyennement satisfait	2
Très satisfait	3

Donc si un individu est "Très satisfait", alors $r_{if} = 3$.

- Remplacer le rang de chaque variable par une valeur dans $[0,1]$ en remplaçant la variable f dans l'objet par : **NORMALISATION ENTRE 0 ET 1 POUR EGALISER LES POIDS DES VARIABLES**

- $z_{if} = (r_{if} - 1) / (M_f - 1)$

M_f = nombre total de modalités de la variable f
 r_{if} = rang de l'objet i pour cette variable

- 5. Exemple concret

Imaginons une variable "Niveau de satisfaction" avec 3 modalités :

1. Pas satisfait
2. Moyennement satisfait
3. Très satisfait

Alors $M_f = 3$.

Si une personne est "Moyennement satisfaite" :

$$r_{if} = 2$$
$$z_{if} = \frac{2 - 1}{3 - 1} = \frac{1}{2} = 0,5$$

- Elle est donc représentée par 0,5 sur cette variable.

- **Utiliser une distance (Euclidienne ou Manhattan) pour calculer la similarité**

Quand on combine plusieurs variables ?

Exemple : $O1(X1,X2,X3,Y1,Y2)$ et $O2(X1,X2,X3,Y1,Y2)$ où $X1,X2,X3$ sont des données **numériques** et $Y1,Y2$ **binaires** ;

→ Comment calculer la dist(O1,O2) ?

- Etape 1 : Je compare $O1$ et $O2$ en fonction de $X1, X2$ et $X3$
 - Ex : $\text{Manhattan}(O1,O2)_{X1,X2,X3} = M$
- Etape 2 : Je compare $O1$ et $O2$ en fonction de $Y1$ et $Y2$
 - Ex : $\text{Jaccard}(O1,O2)_{Y1,Y2} = J$
 - Comme M est **beaucoup plus grand** que J (qui lui n'a que comme données 0 et 1), faire juste $J+M$ pour comparer $O1$ et $O2$ revient à ne prendre en compte que les variables **numériques**
 - Même souci avec les variables **ordinales** -> **On doit les normaliser pour que toutes les variables aient les mêmes poids.**
 - Il va donc falloir **normaliser** d'abord M en la **divisant** par la **distance maximale (Mmax)** en considérant les **valeurs numériques**.
 - On va créer un **Omin** et un **Omax (objets fictifs)** qui vont prendre réciproquement les valeurs minimales et maximales sur chacune des variables numériques (ici 3)

La distance maximale Mmax est obtenue en calculant la distance entre Omin et Omax, qui sont deux objets obtenus comme suit :

Omin : On affecte à $X1, X2$ et $X3$ les valeurs minimales de $X1, X2$ et $X3$

Omax : On affecte à $X1, X2$ et $X3$ les valeurs maximales de $X1, X2$ et $X3$

La distance entre $Omin$ et $Omax$ sera donc la distance **maximale**.

Donc :

- $M(O1,O2)/Mmax \leq 1$ donc c'est comparable à J (qui est aussi entre 0 et 1)
 - **On peut donc comparer $O1$ et $O2$ en calculant $(M(O1,O2)/Mmax) + J(O1,O2)$**

V- Similarité entre objets – calcul de distance entre O1 et O2

$$\text{Distance}(O1, O2) = 1/\text{Similarité}(O1, O2)$$

(Plus la distance est petite, plus la similarité est grande)

Comment calculer la distance ?

On va utiliser une **fonction de distance** qui va **quantifier l'écart** entre O1 et O2 selon leurs caractéristiques.

Une formule de distance courante est la distance de **Minkowski** :

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)}$$

Tel que :

- *i* est un objet O1 qui contient **p caractéristiques** (ex : âge, salaire, ancienneté...)
 - $i = (x_{i1}, x_{i2}, \dots, x_{ip})$
 - **NB** : x_{i1} ne représente pas le même attribut que x_{i2} , l'un est l'âge par exemple et l'autre le salaire...
 - Ex d'objet O1 :

Personne	Âge	Salaire	Ancienneté
i	25 ans	2 000 €	1 an

$$i = (25, 2000, 1)$$

- *j* est un objet O2 qui contient **p caractéristiques** (ex : âge, salaire, ancienneté...)
 - $j = (x_{j1}, x_{j2}, \dots, x_{jp})$
- **q** est un entier positif (qui va déterminer le **type de distance**)
 - **Si q=1**, alors d correspond à la **distance de Manhattan**

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

On additionne les écarts absolus sur chaque variable

- **Si q=2**, alors d correspond à la **distance Euclidienne**

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ip} - x_{jp})^2}$$

Propriétés :

- $d(i, j) \geq 0$
- $d(i, i) = 0$
- $d(i, j) = d(j, i)$
- $d(i, j) < d(i, k) + d(k, j)$

Exemple : distance de Manhattan (q=1)

	Age	Salaire
Personne1	50	11000
Personne2	70	11100
Personne3	60	11122
Personne4	60	11074

Exemple de $d(p1,p2)$:

$$d(p1,p2) = |50-70| + |11000-11100| = 20+100 = \mathbf{120}$$

Autre exemple de $d(p1,p3)$:

$$d(p1,p3) = |50-60| + |11000-11122| = \mathbf{132}$$

➔ **CONCLUSION** : p1 ressemble plus à p2 qu'à p3 (plus la distance est petite, plus elle est similaire)

○ **PROBLEME** 🙄

➔ MAIS vu qu'on n'a pas standardisé nos différents éléments, c'est compliqué de les comparer d'un point de vue statistique, donc vaut mieux les **standardiser**.

Reprise de l'exemple de Distance de Manhattan (q=1) avec standardisation :

	Age	Salaire
Personne1	-2	-2
Personne2	2	0.70
Personne3	0	1.3
Personne4	0	0

Exemple de $d(p1,p2)$:

$$d(p1,p2) = |-2-2| + |-2-0.7| = 4 + 2.7 = \mathbf{6.7}$$

Autre exemple de $d(p1,p3)$:

$$d(p1,p3) = |-2-0| + |-2-1.3| = \mathbf{2+3.3 = 5.3}$$

➔ **CONCLUSION** : p1 ressemble plus à p3 qu'à p2 (plus la distance est petite, plus elle est similaire)

VI- Approches de Clustering

1. Algorithmes de Partitionnement

Ces algorithmes **divisent les données en plusieurs groupes** (ou partitions).

Chaque objet appartient à **un seul cluster**.

Ensuite, on **évalue la qualité des regroupements** selon certains critères (comme la distance moyenne intra-cluster).

Exemple : **K-means** est un algorithme de partitionnement classique.

a. Fonctionnement de l'algorithme de partitionnement

On crée tout d'abord une **partition** à **k clusters** d'une **base D** à **n objets**.

Ensuite, les **k clusters** créées doivent **optimiser** le **critère** choisi.

Deux approches existent :

- **Global Optimal** : Examiner toutes les partitions possibles, afin de trouver la meilleure selon le critère choisi.
- **Heuristics Methods** : Elles cherchent une bonne solution en utilisant des algorithmes rapides et efficaces (ex : k-means ou k-medoids)

b. Méthode des k-moyennes (K-means)

Etape 1 : Choisir nos **k objets** qui vont ainsi former **k clusters**.

→ Chaque objet va servir de centre initial de cluster **appelé M_i**

Etape 2 : Répéter jusqu'à stabilisation (jusqu'à ce qu'il n'y ait plus de changements possibles) :

Etape 2.1 : Pour chaque objet **O**, on calcule la distance entre **O** et tous les centres de clusters **M_i** .

Etape 2.2 : On attribue **O** au cluster dont le centre est le **plus proche (distance minimale)**

Etape 2.3 : Recalcul des centres **M_i** pour chaque cluster (le **barycentre**) qui va correspondre à la moyenne des coordonnées de tous les objets du cluster

Exemple : si ton cluster a trois points **(1,2)** , **(2,3)** , **(3,4)** , le nouveau centre sera **$((1+2+3)/3, (2+3+4)/3) = (2,3)$** .

i. Exemple

Données :

Soit A un ensemble de points tel que $A = \{1, 2, 3, 6, 7, 8, 13, 15, 17\}$ et tel que l'on a 3 clusters ($k = 3$)

Parmi A, on prend 3 objets aux hasards (un par cluster) -> Ici, ça sera 1, 2 et 3

→ Ce qui donne :

- $C1=\{1\}$ avec le centre $M1=1$
- $C2=\{2\}$ avec le centre $M2=2$
- $C3=\{3\}$ avec le centre $M3=3$

Etape 1 : Première affectation

Chaque objet de A (autre que 1 2 et 3) est affecté au cluster dont le centre est le plus proche.

Exemple avec 6 :

On va comparer 6 avec M1, M2 et M3 :

- $dist(6, M1) = |6-1| = 5$
- $dist(6, M2) = |6-2| = 4$
- $dist(6, M3) = |6-3| = 3 \rightarrow 6$ va donc dans **C3**

Après avoir ajouté tous les points selon cette règle :

- $C1 = \{1\}, M1 = 1$
- $C2 = \{2\}, M2 = 2$
- $C3 = \{3, 6, 7, 8, 13, 15, 17\}, M3 = (3+6+7+8+13+15+17)/7 = 69/7 \approx 9.86$

Etape 2 : Recalcul des centres et réaffectation

On vérifie pour **chaque point** si un autre centre est plus proche que celui actuel.

→ **Tous les points dans tous les clusters sont concernés.**

Exemple avec 3 dans C3 :

- $dist(3, M1) = |3-1| = 2$
- $dist(3, M2) = |3-2| = 1$
- $dist(3, M3) = |3-9.86| = 6.86$

Donc 3 passe en **C2** -> $C2=\{2,3\} \rightarrow M2 = (2+3)/2 = 2.5$

Etc... Jusqu'à que plus aucun point ne change de cluster. **STABILISATION FINALE**

ii. Forces de K-Means

- Rapide et efficace : $O(tkn)O(tkn)O(tkn)$
- Crée des clusters compacts en minimisant la distance intra-cluster

iii. Faiblesses de K-Means

- Nécessite des données numériques (moyennes)
- Il faut choisir k à l'avance
- Les centres peuvent ne pas correspondre à des points réels
- Ne détecte pas les clusters non-convexes
- Sensible aux outliers

2. Algorithmes Hiérarchiques

Ces algorithmes créent une **hiérarchie de regroupements** selon certains critères.

On peut les visualiser sous forme d'un **arbre** (appelé dendrogramme).

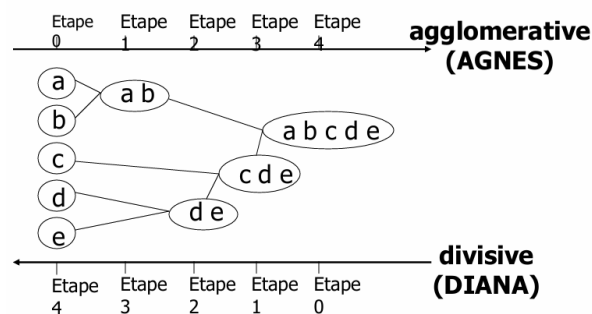
Les **objets proches** sont d'abord **regroupés**, puis les **groupes** sont **fusionnés** progressivement selon un **critère de proximité**.

Exemple : le clustering hiérarchique agglomératif.

On va utiliser la matrice de distances entre objets pour séparer ou regrouper les points.

➔ Contrairement à k-means, pas besoin de préciser le nombre de clusters

- MAIS on a besoin d'une **condition d'arrêt**



2 types de clustering hiérarchique :

- Agglomératif (AGNES) (on va se focus sur lui)

- On commence avec chaque objet comme cluster **individuel**
- Puis on fusionne les clusters étape par étape
- A la fin tout est dans un seul cluster
- Chaque étape correspond à une fusion, jusqu'à ce qu'il ne reste qu'un cluster
- Pour les méthodes agglomératives, C1 et C2 sont fusionnés si :
 - Règle 1 : Si au moins un couple d'objets (un dans chaque cluster) est suffisamment proche (inférieur au seuil), alors on fusionne les clusters.
 - On fusionne C1 et C2 s'il existe o1 dans C1 et o2 dans C2 tels que $\text{dist}(o1, o2) \leq \text{seuil}$.
 - Règle 2 : Si tous les objets de C1 et C2 sont à une distance inférieure au seuil, alors on peut fusionner.
 - On fusionne C1 et C2 si il n'existe pas o1 dans C1 et o2 dans C2 tels que $\text{dist}(o1, o2) \geq \text{seuil}$.
 - Règle 3 : On peut aussi fusionner si la distance moyenne entre tous les couples d'objets de C1 et C2 est inférieure au seuil.
 - Distance moyenne entre clusters $\leq \text{seuil}$:

$$\text{dist}(C_1, C_2) = \frac{1}{n_1 \cdot n_2} \sum_{o_1 \in C_1} \sum_{o_2 \in C_2} \text{dist}(o_1, o_2) \leq \text{seuil}$$

où :

- $n_1 = |C_1|$ et $n_2 = |C_2|$ sont les nombres d'objets dans les clusters C1 et C2.
- o_1 parcourt tous les objets de C1, o_2 parcourt tous les objets de C2.
- $\text{dist}(o_1, o_2)$ est la distance entre les objets o_1 et o_2 .

- Divisif (DIANA)

- L'inverse
-

3. Algorithmes basés sur la Densité : DBscan

Ces algorithmes **regroupent les objets** selon leur **densité dans l'espace des données**.

L'idée est que les **objets d'un même cluster** sont **proches** et **denses**, tandis que les zones de faible densité représentent du **bruit**.

Exemple : DBSCAN est un algorithme basé sur la densité.

a. Paramètres clés

- **Eps** : Rayon maximum du voisinage autour d'un point (distance pour que l'on puisse considérer deux points comme voisins. Pour chaque point p , on regarde tous les points qui sont à une distance **inférieure** à **Eps** -> Pour déterminer qu'ils sont voisins.

Formule :

$$V_{\text{Eps}}(p) = \{q \in D \mid \text{dist}(p, q) \leq \text{Eps}\}$$

où $V_{\text{Eps}}(p)$ est le voisinage de p .

Exemple concret :

- On a un point p et $\text{Eps} = 2$.
- Tous les points à distance ≤ 2 de p font partie de son voisinage.
- Si un autre point q est à distance 1.5 de p , alors $q \in V_{\text{Eps}}(p)$.
- Si un point r est à distance 2.5, alors $r \notin V_{\text{Eps}}(p)$.

○ Rôles :

- Détermine la **taille locale** de la zone qu'on considère comme **dense**
- **Trop petit** -> Beaucoup de **points isolés** (clusters minuscules ou beaucoup de bruit)
- **Trop grand** -> **Clusters trop gros**, risque de fusionner des zones qui ne devraient pas l'être

- **MinPts** : Nombre minimum de points dans le **voisinage-eps** d'un point pour que ce point soit considéré comme **attractif**.

○ **Attractif** : Point autour duquel il y'a une densité suffisante de points

Exemple concret :

- $\text{MinPts} = 5$
- Pour que p soit attractif : il faut que p et au moins 4 autres points soient dans $V_{\text{Eps}}(p)$.
- Si seulement 3 points sont dans son voisinage, p ne sera pas attractif et sera peut-être considéré comme **bordure** ou **bruit**.

○ Rôles :

- Contrôle la densité minimale nécessaire pour former un cluster
- **Trop petit** -> Zones clairsemées deviennent des clusters -> bruit mal détecté
- **Trop grand** -> Seules les zones très denses sont détectées -> Beaucoup de points deviennent bruit.

Un point q est directement densité-accessible depuis p si :

1. p est proche de q (dans son rayon Eps), et
2. q est un **point attractif** (entouré d'au moins MinPts points dans son voisinage).

En d'autres termes : p peut rejoindre q dans le cluster si q est suffisamment dense et proche.

Comment DBscan relie-t-il les points de E à partir d'un graphe G pour former des clusters ?

A. Construction du graphe G

- On considère l'ensemble de points E sur lequel on veut appliquer DBSCAN.
- On construit un **graphe G** :
 - **Sommets** = chaque point de E
 - **Arêtes** (liens entre points) : deux points p et q sont reliés si :
 1. Ils sont voisins $\rightarrow \text{dist}(p,q) < \text{Eps}$
 2. Au moins l'un des deux est attractif (entouré d'au moins MinPts points)

B. Formation des clusters via le graphe

- Une fois le graphe construit, les clusters correspondent aux composantes connexes du graphe.
 - **Composante connexe** : Ensemble de sommets reliés entre eux par des arêtes.
- Autrement dit : deux points font partie du même cluster si on peut "marcher" d'un point à l'autre en suivant les arêtes.

Exemple intuitif :

- Imagine des points reliés par des cordes :
 - Tu peux passer de p à q **en restant sur les cordes** $\rightarrow$ p et q sont dans le même cluster.
 - Si aucun chemin ne relie p et q $\rightarrow$ ils sont dans des clusters différents ou du bruit.

C. Particularités de DBSCAN

1. Points très éloignés dans l'espace peuvent être dans le même cluster si **une chaîne de points attractifs les relie**.
2. Les clusters peuvent être **non convexes** (formes libres), contrairement à K-Means qui produit toujours des clusters ronds ou sphériques.
3. Le graphe permet de **propager les clusters à partir des points attractifs**, en ajoutant tous les voisins accessibles densément.